

Правительство Российской Федерации Федеральное государственное
автономное образовательное учреждение высшего образования

Национальный исследовательский университет

«Высшая школа экономики»

Факультет компьютерных наук

Образовательная программа

«Дизайн и разработка информационных продуктов»

ПРОМЕЖУТОЧНЫЙ ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ

На тему «Платформа для no-code автоматизации бизнес процессов»

Студента 2 курса

группы БДРИП241

Гаврилина Станислава Васильевича

Студента 2 курса

группы БДРИП242

Клушина Артёма Алексеевича

Научный руководитель

Шкель Алексей Алексеевич

Москва, 2026

Содержание

Описание проекта к контрольной точке 1	3
1. Основные планы и этапы проекта	3
1.1. Краткое описание проекта	3
1.2. Планы и этапы выполнения проекта	3
2. Используемый технологический стек и его обоснование	5
2.1. Перечень используемых технологий	5
2.2. Обоснование выбранного технологического стека	8
3. Критерии оценивания проекта	8
Источники	11

Описание проекта к контрольной точке 1

1. Основные планы и этапы проекта

1.1. Краткое описание проекта

Название проекта: Платформа для no-code автоматизации бизнес процессов

Цель проекта: построить собственный движок исполнения workflow с отказоустойчивостью и масштабированием.

Краткое описание задач: Промышленный Java-движок исполнения графов с отказоустойчивостью и горизонтальным масштабированием, совместимый по концепции с p8n (узлы/связи), но с версионированием, упрощённым debug-ом, snapshot-ами, фича-флагами

1.2. Планы и этапы выполнения проекта

Таблица 1.1. Планы и этапы выполнения проекта

Этап проекта	Описание работ	Ожидаемые результаты	Сроки выполнения
Проектирование архитектуры	Определение структуры системы, проектирование модели данных для workflow, выбор подходов к хранению состояния	Архитектурная документация, ER-диаграмма, схема взаимодействия компонентов	Декабрь 2025

Этап проекта	Описание работ	Ожидаемые результаты	Сроки выполнения
Редактор workflow	Реализация визуального редактора: canvas для размещения нод, создание связей между нодами, сохранение и загрузка workflow	Веб-интерфейс для создания и редактирования графа workflow	Январь 2026
Движок исполнения (базовый)	Реализация ядра исполнения графа, поддержка триггеров (ручной запуск, scheduler, webhook), базовый HTTP-узел	Работающий движок, способный исполнять линейные workflow с HTTP-запросами	Февраль 2026
Просмотр состояния и отладка	Реализация интерфейса просмотра состояния выполнения: входные/выходные данные каждого узла, ошибки, история запусков	Полный мониторинг исполнения workflow в реальном времени	Март 2026
Расширенные возможности	Потоки данных (filter, map, reduce, foreach, flatmap), Code Node с песочницей Python, параллельное исполнение веток графа	Полнофункциональный движок, соответствующий требованиям MVP	Апрель 2026
Тестирование и документация	Покрытие тестами, оформление курсовой работы	Протестированная система, оформленная документация	Апрель 2026

2. Используемый технологический стек и его обоснование

2.1. Перечень используемых технологий

Технология / инструмент	Описание	Причины выбора
Java/Kotlin	Основной язык программирования для разработки серверной части (Backend).	• Строгая типизация: Обеспечивает надежность кода и упрощает поддержку сложной логики работы с графами. • Многопоточность: Эффективная работа с параллельными процессами и фоновыми задачами (в отличие от однопоточных аналогов). • Экосистема: Огромный выбор готовых библиотек и инструментов для решения любых интеграционных задач.
Spring (Spring Boot)	Фреймворк для создания архитектуры приложения и API.	• Скорость разработки: Готовые модули для работы с БД, безопасностью и веб-запросами ускоряют создание MVP.

		• Управление зависимостями: Упрощает конфигурацию компонентов и поддержку чистоты архитектуры. • Стандарт индустрии: Легкость в поиске решений, документации и специалистов.
PostgreSQL	Реляционная база данных для хранения данных и состояния системы.	• Гибкость данных: Отличная поддержка JSON позволяет хранить структуры воркфлоу и неструктурированные данные в одной БД. • Надежность (ACID): Гарантирует сохранность данных и целостность транзакций при сбоях. • Функциональность: Мощные возможности индексации и масштабирования "из коробки".

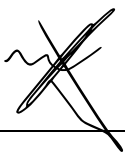
Angular	Фреймворк для разработки клиентской части (Frontend) и визуального редактора.	• Структурированность: Четкая архитектура навязывает порядок в коде, что критично для создания сложных интерфейсов. • TypeScript: Использование типов на клиенте синхронизируется с подходом Java на бэкенде. • Производительность: Оптимизирован для работы с большими и динамичными веб-приложениями.
---------	---	---

2.2. Обоснование выбранного технологического стека

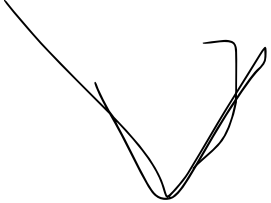
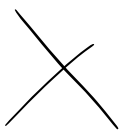
Выбранный технологический стек ориентирован на разработку надежного, масштабируемого и поддерживаемого приложения с четким разделением ответственности между клиентской и серверной частями. Использование Java/Kotlin в сочетании со Spring Boot позволяет реализовать сложную бизнес-логику и работу с графовыми структурами, обеспечивая строгую типизацию, предсказуемость поведения системы и высокую устойчивость к ошибкам. PostgreSQL дополняет серверную часть за счёт транзакционной надежности и гибкости хранения данных, что особенно важно при работе с состоянием воркфлоу и частично структурированными данными.

Для клиентской части выбран Angular как фреймворк, обеспечивающий строгую архитектурную модель и единый подход к разработке за счёт использования TypeScript. Это позволяет снизить когнитивную нагрузку при разработке, повысить качество кода и упростить синхронизацию моделей данных между фронтендом и бэкендом. В совокупности данный стек соответствует индустриальным стандартам, хорошо подходит для создания MVP и обеспечивает задел для дальнейшего развития и масштабирования системы.

3. Критерии оценивания проекта

Критерий	Описание
Использование инструмента автоматической документации API	Будут использованы 
Использование адаптивного дизайна	Будут использованы 
Работа с базой данных	Будут использованы 
Функциональность - Процент выполнения функциональных требований	Выполненные требования в процентах от общего количества 

Производительность и эффективность - Время отклика	Среднее время ответа программы на запросы пользователя (в секундах) 
Качество кода - Покрытие кода тестами (%)	Test coverage 
Тестирование - Количество написанных тестов	Общее количество юнит-тестов, интеграционных тестов и других видов тестов 
Тестирование - Процент успешных тестов (%)	Процент успешно пройденных тестов из общего количества 
Оценка командной работы - Количество завершенных задач на каждого участника	Общее число задач, выполненных каждым членом команды 

Реализация параллельного выполнения, Поддержка параллельного выполнения веток графа workflow	Будут использованы 
Наличие механизма хранения истории изменений	Будут использованы 
Ограничение доступа пользовательского Python-кода	Наличие sandbox-ограничений. 
Наличие архитектурных диаграмм системы	будут использованы 

Источники

1. Spring Framework Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-framework/reference/> (дата обращения: 03.02.2026).
2. Angular Documentation [Электронный ресурс] / Google. — URL: <https://angular.dev/> (дата обращения: 03.02.2026).
3. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка: пер. с англ. / М. Клеппман. — СПб.: Питер, 2018. — 640 с. — ISBN 978-5-4461-0512-0
4. **PostgreSQL Documentation** [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 03.02.2026).